

Thanapat Aupprathumwipanon

Software Engineer · Full-Stack Developer

Computer Engineering student at Chulalongkorn University. I build systems that other people depend on — a mentorship platform carrying **1,000+ mentors and mentees**, registration flows that absorbed **5,000+ students** on opening day, and a retrieval-augmented reporting API answering to a **Thai government agency's** integration contract.

What follows is the work I would want to be judged on, and the reasoning behind it.

01 Wise Mentoring Ship

02 RAG GIS API — ONEP

03 Tax Tracker

04 First Date & RubpuenKaomai

05 Agaya

06 Also built

01 Wise Mentoring Ship

A cross-university mentorship platform — built at an agency, then handed to me to keep.

ROLE	Software Engineer Intern, Skooldio (Feb – Jul 2026) → Software Engineer (Part-time), Wise (Aug 2026 – present)
SCALE	1,000+ mentors and mentees across Thai universities
TEAM	Co-developed features alongside the engineering manager

Universities had no shared way to pair students with mentors: matching happened in spreadsheets and chat groups, and nobody could see the state of a request once it left the room. Wise Mentoring Ship turns that into one platform — mentees browse and request, mentors accept or decline, and administrators can see and correct the whole pipeline.



WHAT I BUILT

- The registration flow, the mentee-request and mentor-handling logic, the dashboard, the homepage, the paginated explore pages, and the admin back-office flow — front end through to API, on top of the base design tokens and reusable components I set up for the rest of the UI.
- A data-migration plan and the scripts behind it, to move records already in use into the new schema without losing or duplicating a relationship.
- The CI/CD pipeline, and the LINE integration — Official Account, Flex messages and the Developer APIs — with GCP Cloud Tasks driving scheduled and asynchronous notifications.

HOW IT WAS BUILT

Clean architecture, not just clean files

DTOs, Entities and Use Cases kept apart, so a change to the transport layer never reaches into domain logic. Swagger docs generated from the same definitions the API validates against.

Notifications that survive a slow request

Scheduled and fan-out LINE messages run through GCP Cloud Tasks rather than inside the request that triggered them, so a slow push never becomes a failed user action.

STILL MINE

When the Skooldio engagement ended in July, Wise retained me directly to keep owning the backend of what we had built — shipping API and back-office features, resolving production issues against a live user base, and extending into the React front end when a change spans the stack.

React

NestJS

TypeScript

PostgreSQL

GCP Cloud Tasks

LINE OA / Flex

Swagger

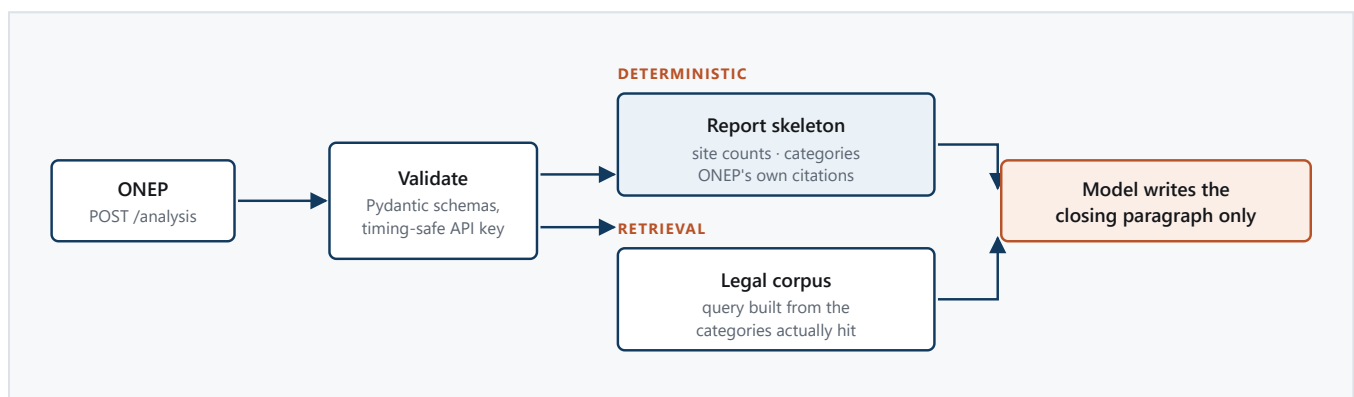
CI/CD

02 RAG GIS API

Turning GIS impact analyses into Thai environmental reports — for a government agency that retries on failure.

ROLE	Contributor — the /analysis endpoint end to end, plus SSE streaming for the chat API
CLIENT	ONEP — Office of Natural Resources and Environmental Policy and Planning
SOURCE	github.com/dannifyz/rag-gis-api

Before a project breaks ground, ONEP runs a spatial analysis: which protected natural sites, cultural sites and conservation zones fall inside the legal radius, how close, how much overlap. The output is structured data. What the agency actually needs is a Thai narrative report that reads those numbers back with the right regulation cited beside them — work that used to be done by hand. This API does it.



THE DECISIONS I WOULD DEFEND IN REVIEW

Numbers cannot be allowed to drift

Every count, category breakdown and legal citation is assembled in code before the model is called. The model is handed that finished skeleton and asked only for a closing synthesis paragraph — so there is no figure in the report a hallucination could move.

Fail the way the caller expects

ONEP retries on 5xx, 429 and timeout, but treats 4xx as permanent rejection. So an empty model response raises rather than returning 200 with a half-written report, and the endpoint budgets 100s inside their 120s disconnect window — a hang becomes a clean retry, not a dropped request.

Forward-compatible on purpose

Unknown fields are ignored and an unrecognised schema_version is logged and processed rather than rejected: a future version of their payload still carries every field we read, and refusing it would break a live integration over a version string.

Retrieval grounded in this project

The RAG query is built from the project type plus only the site categories with a non-zero count, so retrieval is not steered toward law about sites this project never touches.

Also contributed: SSE token streaming for the chat API, so answers render as they are generated rather than after the full response completes, with source listing for the chunks behind each answer.

Python 3.14

FastAPI

Pydantic

Google Gemini

Vector search

Typhoon OCR

SSE

uv

Ruff

03 Tax Tracker

Capital-gains tax for Thai investors, computed from broker PDFs that never leave the browser.

ROLE	Sole author
WHY	Built for my own filing, after doing a year of it by hand in a spreadsheet
SOURCE	github.com/miramslp/tax-tracker

Thai investors holding foreign equities have to report realised capital gains, and their broker gives them confirmation notes as PDFs — one per trade, no export. Reconstructing FIFO cost basis across a year of those by hand is slow and easy to get wrong. Drop the PDFs into the page and it does the accounting.

Screenshot needed

The drop zone with files loaded, or the computed gain/loss table

Screenshot needed

The annual tax summary, or the statement-validation view

THE DECISIONS I WOULD DEFEND IN REVIEW

Refuse rather than guess

The parser reads its broker's layout by character position, and rejects a document it does not recognise instead of inferring structure. A wrong number here goes onto a tax return — a confident guess is worse than a refusal.

The data has nowhere to go

No server, no database, no upload. Parsing runs entirely in the browser and the Content Security Policy is locked to `connect-src 'self'`, so a complete trade history cannot be exfiltrated even by a dependency that wanted to.

Check the arithmetic against a second source

Confirmation notes are cross-checked against monthly statements, so a missing or duplicated trade surfaces as a discrepancy rather than quietly shifting every FIFO lot after it.

Say what it cannot do

Corporate actions — splits and stock dividends — break FIFO cost basis, so the tool warns rather than reporting a clean number it has no basis for. Documented limits, not hidden ones.

TypeScript

Vite

In-browser PDF parsing

FIFO cost basis

CSP

Vercel

04 First Date & RubpuenKaomai 2026

Freshman-welcome registration for 5,000+ students, on a day when everyone arrives at once.

ROLE Frontend Developer — ISD, Chulalongkorn University (Jul 2026)

SCALE 5,000+ students, concentrated into the opening hours of sign-up

Two student-organisation platforms for Chulalongkorn's freshman-welcome events. I built the registration flow, the landing page and the edit-profile UI, and integrated the APIs behind them — the part of the system that has to stay upright while an entire incoming year signs up in the same hour.

Screenshot needed

First Date landing page

Screenshot needed

Registration or edit-profile screen

React

API integration

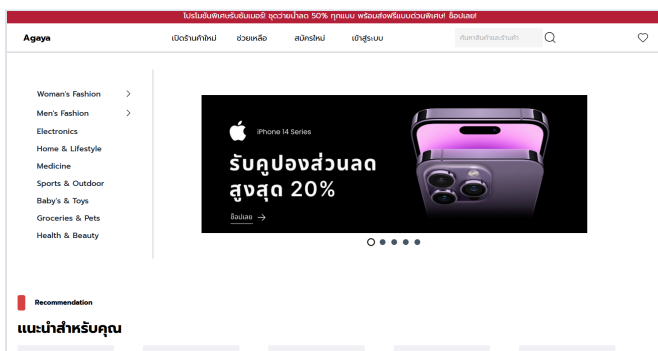
Responsive UI

05 Agaya

A middleman e-commerce platform — and the first project I led rather than only built.

ROLE Team Lead — Agile sprint coordination, full-stack task ownership, end-to-end delivery

SOURCE github.com/Pachara2103/Agaya



A platform sitting between vendors and customers, handling the listing, the order and the money in between. I ran the sprints and owned delivery across the stack, and built the payment and media paths myself: Stripe for processing, Cloudinary for product-image storage and delivery.

Full-stack

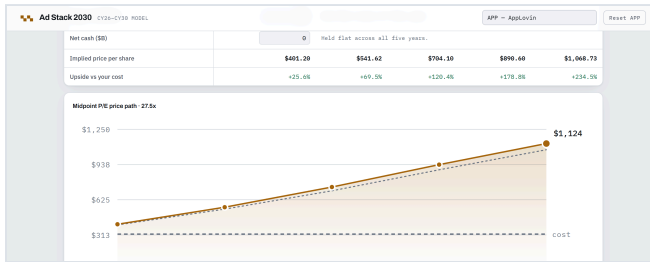
Stripe

Cloudinary

Agile

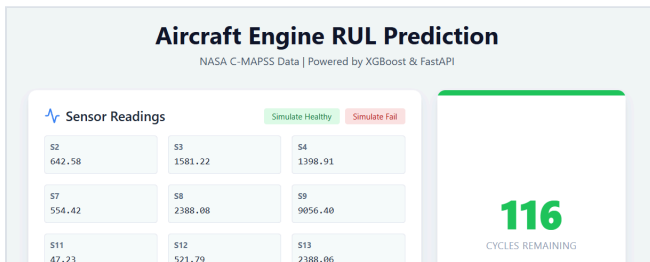
06 Also built

Smaller pieces, each one built to answer a question I actually had.



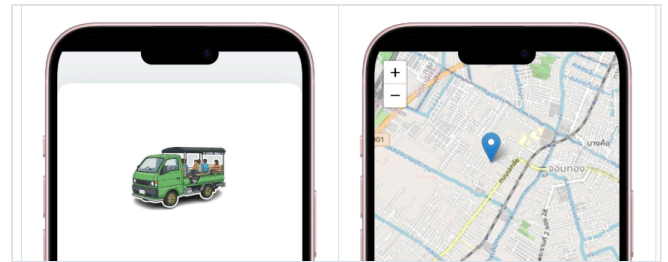
Ad Stack 2030

An interactive equity model covering 37 companies across 8 sectors. Edit the drivers — growth, margin, share count — and every projection, bear/base/bull case, multiple and IRR through 2030 re-runs live in the browser.

[React 19](#)[Vite](#)[Financial modelling](#)

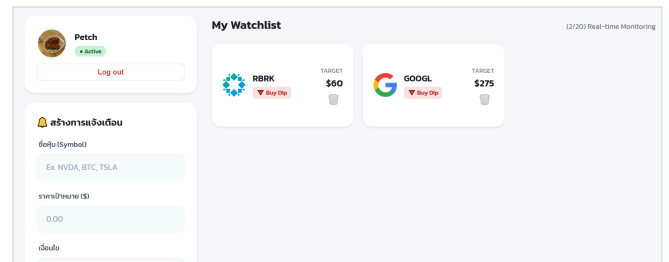
Predictive Maintenance

Predicts remaining useful life of turbofan engines from NASA C-MAPSS sensor data. XGBoost benchmarked against Random Forest and Decision Tree, ending at RMSE 16.78 cycles and $R^2 \sim 0.82$, served through a live dashboard.

[Python](#)[XGBoost](#)[FastAPI](#)[React](#)

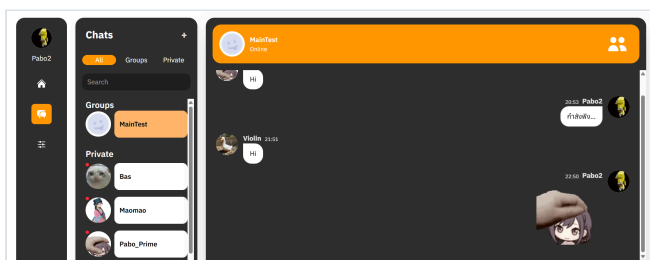
Krapong-Go

Real-time transit for Thonburi, my hometown. Passengers signal demand and drivers see it live on the map, so a driver knows there is someone waiting instead of guessing and running empty.

[Socket.IO](#)[Redis](#)[MongoDB](#)[Docker](#)

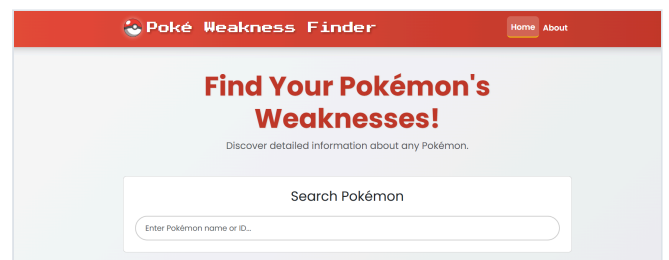
Stock Watcher

Automated price alerts pushed into a LINE group, with LINE Login for auth. Used daily by a private group of friends and family who were otherwise checking prices by hand.

[LINE Messaging API](#)[LINE Login](#)

Otto Webchat

A group chat app, built mainly to learn the deployment half: a load balancer on AWS Elastic Beanstalk and a custom domain in front of it.

[AWS](#)[Elastic Beanstalk](#)[Node.js](#)

Mini-RAG & other experiments

A hybrid BM25-plus-embedding retriever over Thai environmental and arts law, chunked by legal article rather than token count and scored with Hit@k / Recall@k / Precision@k — the groundwork behind project 02. Alongside a VGA image switcher on a Basys3 FPGA, and a Pokémon weakness finder built because no site answered the question fast enough.

[Python](#)[BM25](#)[Embeddings](#)[Verilog](#)